

Preentrega Backend con Java

Jose Miguel Paz Portilla

1 de octubre de 2026

Índice

1. Creación del proyecto	3
1.1. build_new_project.sh	3
2. Ejecución del proyecto	5
2.1. Main.java	6
2.2. Validador.java	6
2.3. Producto.java	9
2.4. ProductoService.java	11
2.5. ProductoNoEncontradoException.java	14

1. Creación del proyecto

Para crear un nuevo proyecto java en linux desde la terminal utilizando el script `build_new_project.sh` el cual crea las carpeta como lo necesita maven, además crea un `Main.java` de muestra y crea las carpetas `ui`, `util`, `model`, `exception` y `service`. Este archivo debe tener permiso de ejecución `chmod +x build_new_project.sh`

1.1. build_new_project.sh

Listing 1: Script de automatización `build_new_project.sh`

```
1  #!/bin/bash
2
3  # Detener el script inmediatamente si ocurre algun error
4  set -e
5
6  # =====
7  # VALIDACION DE ARGUMENTOS DE LINEA DE COMANDO
8  # =====
9  # Si el primer argumento ($1) esta vacio, muestra un error
   explicativo y sale.
10 if [ -z "$1" ]; then
11     echo "Error: No especificaste el nombre del proyecto."
12     echo "Uso correcto: $0 <nombre_del_proyecto>"
13     echo "Ejemplo: $0 preentrega"
14     exit 1
15 fi
16
17 # Asignamos el primer argumento de la terminal a nuestra variable
18 NUM_EJERCICIO="$1"
19
20 # Definicion de rutas estandar de Maven usando la variable
21 PACKAGE_DIR="src/main/java/com/josepaz/$NUM_EJERCICIO"
22 TEST_DIR="src/test/java/com/josepaz/$NUM_EJERCICIO"
23 RESOURCES_DIR="src/main/resources"
24
25 echo "=== 1. Generando estructura de carpetas estandar de Maven y
   paquetes internos para: $NUM_EJERCICIO ==="
26 # Creamos la ruta base del paquete y del entorno de pruebas
27 mkdir -p "$PACKAGE_DIR"
28 mkdir -p "$TEST_DIR"
29 mkdir -p "$RESOURCES_DIR"
30
31 # Creamos las subcarpetas de arquitectura dentro de la carpeta del
   ejercicio
32 mkdir -p "$PACKAGE_DIR/exception"
33 mkdir -p "$PACKAGE_DIR/model"
34 mkdir -p "$PACKAGE_DIR/service"
35 mkdir -p "$PACKAGE_DIR/util"
36 mkdir -p "$PACKAGE_DIR/ui"
37
38 echo "=== 2. Verificando archivo Java principal ==="
```

```

39 # Solo crea el Main.java de ejemplo inicial si el archivo NO existe
40 .
41 if [ ! -f "$PACKAGE_DIR/Main.java" ]; then
42     echo "Creando archivo Java de ejemplo (Main.java)..."
43     cat << EOF > "$PACKAGE_DIR/Main.java"
44 package com.josepaz.$NUM_EJERCICIO;
45
46 public class Main
47 {
48     public static void main(String[] args)
49     {
50         System.out.println("Hola Mundo desde Main.java en Maven (
51             $NUM_EJERCICIO)");
52     }
53 }
54 EOF
55 else
56     echo "El archivo Main.java ya existe. Respetando tu codigo
57         actual."
58 fi
59
60 echo "=== 3. Generando archivo pom.xml ==="
61 cat << EOF > pom.xml
62 <?xml version="1.0" encoding="UTF-8"?>
63 <project xmlns="http://apache.org"
64     xmlns:xsi="http://w3.org"
65     xsi:schemaLocation="http://apache.org http://apache.org">
66     <modelVersion>4.0.0</modelVersion>
67
68     <groupId>com.josepaz</groupId>
69     <artifactId>hola-mundo-terminal</artifactId>
70     <version>1.0-SNAPSHOT</version>
71
72     <properties>
73         <maven.compiler.source>21</maven.compiler.source>
74         <maven.compiler.target>21</maven.compiler.target>
75         <project.build.sourceEncoding>UTF-8</project.build.
76             sourceEncoding>
77     </properties>
78
79     <build>
80         <plugins>
81             <!-- Plugin para compilar con tu version Java 21 -->
82             <plugin>
83                 <groupId>org.apache.maven.plugins</groupId>
84                 <artifactId>maven-compiler-plugin</artifactId>
85                 <version>3.13.0</version>
86                 <configuration>
87                     <source>21</source>
88                     <target>21</target>
89                 </configuration>
90             </plugin>

```

```

88         <!-- Plugin Exec apuntando a tu clase dinamica com.
            josepaz.$NUM_EJERCICIO.Main -->
89     <plugin>
90         <groupId>org.codehaus.mojo</groupId>
91         <artifactId>exec-maven-plugin</artifactId>
92         <version>3.5.0</version>
93         <configuration>
94             <mainClass>com.josepaz.${NUM_EJERCICIO}.Main</
                mainClass>
95         </configuration>
96     </plugin>
97 </plugins>
98 </build>
99 </project>
100 EOF
101
102 echo "=== 4. Limpiando y compilando el proyecto ==="
103 mvn clean compile
104
105 echo "=== 5. Ejecutando la clase Main de $NUM_EJERCICIO ==="
106 mvn exec:java

```

2. Ejecución del proyecto

Listing 2: Script de ejecucion run_proyect.sh

```

1  #!/bin/bash
2
3  # Detener el script inmediatamente si ocurre algun error
4  set -e
5
6  # Verificamos si pasaste el argumento por la terminal
7  if [ -z "$1" ]; then
8      echo "Error: No especificaste el proyecto a ejecutar."
9      echo "Uso correcto: $0 <nombre_del_proyecto>"
10     echo "Ejemplo: $0 preentrega"
11     exit 1
12 fi
13
14 # Asignamos el argumento a una variable clara
15 EJERCICIO="$1"
16
17 echo "=== Ejecutando la aplicacion con Maven ($EJERCICIO) ==="
18 mvn package exec:java -Dexec.mainClass="com.josepaz.${EJERCICIO}.
    Main"

```

```
josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM:~/JAVA/java-preentrega$ ./build_new_proyect.sh preentrega
=== 1. Generando estructura de carpetas estándar de Maven y paquetes internos para: preentrega ===
=== 2. Verificando archivo Java principal ===
Creando archivo Java de ejemplo (Main.java)...
=== 3. Generando archivo pom.xml ===
=== 4. Limpiando y compilando el proyecto ===
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.josepaz:hola-mundo-terminal >-----
[INFO] Building hola-mundo-terminal 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ hola-mundo-terminal ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ hola-mundo-terminal ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 0 resource
[INFO]
[INFO] --- maven-compiler-plugin:3.13.0:compile (default-compile) @ hola-mundo-terminal ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 1 source file with javac [debug target 21] to target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 2.502 s
[INFO] Finished at: 2026-10-01T00:59:38-03:00
[INFO]
=== 5. Ejecutando la clase Main de preentrega ===
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.josepaz:hola-mundo-terminal >-----
[INFO] Building hola-mundo-terminal 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec-maven-plugin:3.5.0:java (default-cli) @ hola-mundo-terminal ---
¡Hola Mundo desde Main.java en Maven (preentrega)!
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.874 s
[INFO] Finished at: 2026-10-01T00:59:42-03:00
[INFO]
josepaz@josepaz-Lenovo-IdeaPad-S145-15IGM:~/JAVA/java-preentrega$
```

Figura 1: Creación de un nuevo proyecto java

2.1. Main.java

Listing 3: Clase Main.java (Pre-entrega)

```
1 package com.josepaz.preentrega;
2
3 import util.*;
4
5 public class Main
6 {
7     public static void main(String[] args)
8     {
9         System.out.println("Hola Mundo desde Main.java en Maven (
10             preentrega)");
11         Validador.validarNombre("jose");
12     }
13 }
```

2.2. Validador.java

Listing 4: Clase Validador.java (Pre-entrega)

```

1 package util;
2
3 import java.util.InputMismatchException;
4 import java.util.Scanner;
5
6 import exception.StockInsuficienteException;
7
8 /**
9  * Clase de utilidad que contiene metodos de validacion y lectura
10  * reutilizables.
11  *
12  * Todos los metodos son estaticos: no se requiere crear una
13  * instancia de
14  * Validador para utilizarlos, sino que se invocan directamente
15  * desde la clase.
16  */
17 public class Validador
18 {
19     //
20     // =====
21
22     // VALIDACIONES DE DATOS DEL PRODUCTO
23     //
24     // =====
25
26     // Estos metodos lanzan una excepcion si el dato recibido es
27     // invalido.
28     // No retornan ningun valor: si la ejecucion finaliza sin
29     // lanzar la
30     // excepcion, significa que el dato es valido.
31
32     public static void validarNombre(String nombre)
33     {
34         // Un nombre nulo o vacio no representa un producto valido
35         if (nombre == null || nombre.trim().isEmpty())
36         {
37             throw new IllegalArgumentException("El nombre no puede
38             estar vacio.");
39         }
40     }
41
42     public static void validarPrecio(double precio)
43     {
44         // El precio no debe ser negativo, pero se acepta un valor
45         // de 0
46         if (precio < 0)
47         {
48             throw new IllegalArgumentException("El precio no puede
49             ser negativo.");
50         }
51     }
52 }

```

```

42 public static void validarStock(int stock) {
43     // Un stock negativo no es valido. Se utiliza una excepcion
      personalizada.
44     if (stock < 0)
45     {
46         throw new StockInsuficienteException("El stock no puede
      ser negativo.");
47     }
48 }
49
50 public static void validarCategoria(String categoria)
51 {
52     if (categoria == null || categoria.isBlank())
53     {
54         throw new IllegalArgumentException("La categoria no
      puede estar vacia.");
55     }
56 }
57
58 //
      =====
59 // METODOS DE LECTURA POR CONSOLA
60 //
      =====
61
62 public static int leerEntero(Scanner sc, String mensaje)
63 {
64     // Bucle que se repite hasta que el usuario ingresa un
      entero valido.
65     while (true)
66     {
67         System.out.println(mensaje);
68         try
69         {
70             int valor = sc.nextInt();
71             sc.nextLine(); // Limpia el salto de linea
      pendiente en el buffer
72             return valor;
73         }
74         catch (InputMismatchException e)
75         {
76             System.out.println("Debe ingresar un numero entero.
      Intente nuevamente.");
77             sc.nextLine(); // Limpia el buffer ante un ingreso
      incorrecto
78         }
79     }
80 }
81
82 public static double leerDouble(Scanner sc, String mensaje)
83 {

```

```

84         // Bucle que se repite hasta que el usuario ingresa un
           decimal valido.
85     while (true)
86     {
87         System.out.println(mensaje);
88         try
89         {
90             double valor = sc.nextDouble();
91             sc.nextLine(); // Limpia el salto de linea
                           pendiente en el buffer
92             return valor;
93         }
94         catch (Exception e)
95         {
96             System.out.println("Debe ingresar un numero decimal
                           usando el separador correcto.");
97             sc.nextLine(); // Limpia el buffer ante un ingreso
                           incorrecto
98         }
99     }
100 }
101
102 public static String leerTexto(Scanner sc, String mensaje)
103 {
104     // Lectura simple de una linea de texto completa
105     System.out.println(mensaje);
106     return sc.nextLine();
107 }
108 }

```

2.3. Producto.java

Listing 5: Clase Producto.java (Pre-entrega)

```

1 package model;
2
3 /**
4  * Modelo de dominio: representa un producto del catalogo.
5  *
6  * Aplica encapsulamiento: los atributos son privados y se accede
7  * a ellos a traves de getters y setters. Esta clase no sabe nada
8  * sobre como se almacenan los productos ni como se muestran al
9  * usuario; su unica responsabilidad es representar un producto.
10 */
11 public class Producto
12 {
13     // Atributos privados: nadie desde afuera puede modificarlos
14     // directamente. Para acceder o modificarlos se usan los
       metodos
15     // getters y setters definidos mas abajo.
16     private int id;
17     private String nombre;
18     private double precio;

```

```

19     private int stock;
20     private String categoria;
21
22     // Constructor sin id: el id lo asigna el ProductoService al
23     // momento de guardar el producto. El usuario nunca elige el id
24     public Producto(String nombre, double precio, int stock, String
        categoria)
25     {
26         this.nombre = nombre;
27         this.precio = precio;
28         this.stock = stock;
29         this.categoria = categoria;
30     }
31
32     // Constructor vacio: util para crear un Producto y completarlo
33     // con setters despues. Tambien lo necesitara Spring/JPA mas
        adelante en el curso.
34     public Producto()
35     {
36         //Vacio
37     }
38
39     // Getters y setters: la unica forma de acceder o modificar
40     // los atributos privados desde afuera de la clase.
41     public int getId()
42     {
43         return id;
44     }
45
46     public void setId(int id)
47     {
48         this.id = id;
49     }
50
51     public String getNombre()
52     {
53         return nombre;
54     }
55
56     public void setNombre(String nombre)
57     {
58         this.nombre = nombre;
59     }
60
61     public double getPrecio()
62     {
63         return precio;
64     }
65
66     public void setPrecio(double precio)
67     {
68         this.precio = precio;

```

```

69     }
70
71     public int getStock()
72     {
73         return stock;
74     }
75
76     public void setStock(int stock)
77     {
78         this.stock = stock;
79     }
80
81     public String getCategoria()
82     {
83         return categoria;
84     }
85
86     public void setCategoria(String categoria)
87     {
88         this.categoria = categoria;
89     }
90
91     // toString() sobrescribe el metodo heredado de Object.
92     // Sirve para mostrar el producto de forma legible al listarlo
93     // en consola. Cuando hagamos System.out.println(producto),
94     // llama automaticamente a este metodo.
95     @Override
96     public String toString()
97     {
98         return "ID: " + id +
99             " | " + nombre +
100             " | $" + precio +
101             " | Stock: " + stock +
102             " | Categoria: " + categoria;
103     }
104 }

```

2.4. ProductoService.java

Listing 6: Clase ProductoService.java (Pre-entrega)

```

1 package service;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import exception.ProductoNoEncontradoException;
7 import model.Producto;
8 import util.Validador;
9
10 /**

```

```

11  * Capa de servicio: contiene la logica de negocio de nuestro
    * sistema.
12  *
13  * Es responsable de:
14  * - Mantener la coleccion de productos.
15  * - Asignar el ID al guardar un nuevo producto.
16  * - Validar los datos antes de guardar o actualizar.
17  * - Buscar, modificar y eliminar productos por ID.
18  *
19  * No contiene Scanner ni System.out: no interactua con el usuario.
20  * Quien requiera mostrar mensajes o leer datos lo realiza por
    * fuera (desde la clase Main).
21  * Nota: Esta separacion permitira, en etapas posteriores,
    * reemplazar el menu
22  * por una API REST sin necesidad de modificar este archivo.
23  */
24  public class ProductoService
25  {
26      // Coleccion en memoria que almacena los productos
27      private List<Producto> productos = new ArrayList<>();
28
29      // Contador para asignar IDs unicos. Es static porque pertenece
        a la clase
30      // y no a una instancia, garantizando su unicidad.
31      private static int contadorId = 1;
32
33      //
        =====
34      // OPERACIONES CRUD (Create, Read, Update, Delete)
35      //
        =====
36
37      // CREATE: agregar un nuevo producto
38      public Producto guardar(Producto p)
39      {
40          // Validamos antes de guardar. Si ocurre un error, se lanza
            una
41          // excepcion y el producto NO se agrega a la lista.
42          Validador.validarNombre(p.getNombre());
43          Validador.validarPrecio(p.getPrecio());
44          Validador.validarStock(p.getStock());
45          Validador.validarCategoria(p.getCategoria());
46
47          // El ID lo asigna el servicio, no el usuario.
48          // Luego de asignarlo, se incrementa el contador.
49          p.setId(contadorId);
50          contadorId++;
51
52          // Guardamos el producto en la coleccion
53          productos.add(p);
54

```

```

55         return p;
56     }
57
58     // READ: devuelve toda la lista de productos
59     public List<Producto> listarTodos()
60     {
61         return productos;
62     }
63
64     // READ (Individual): busca un producto por ID
65     public Producto obtenerPorId(int id)
66     {
67         for (Producto p : productos)
68         {
69             if (p.getId() == id)
70             {
71                 return p;
72             }
73         }
74
75         throw new ProductoNoEncontradoException("No se encontro un
76             producto con el ID " + id);
77     }
78
79     // UPDATE: actualiza los datos de un producto existente
80     public Producto actualizar(int id, Producto datos)
81     {
82         // Reutilizamos obtenerPorId. Si lanza excepcion, la
83         // actualizacion se cancela.
84         Producto p = obtenerPorId(id);
85
86         // Validamos los datos nuevos antes de aplicarlos
87         Validador.validarNombre(datos.getNombre());
88         Validador.validarPrecio(datos.getPrecio());
89         Validador.validarStock(datos.getStock());
90         Validador.validarCategoria(datos.getCategoria());
91
92         // Modificamos el producto encontrado.
93         // Como Java maneja los objetos por referencia, los cambios
94         // se reflejan
95         // en la lista sin necesidad de realizar ninguna accion
96         // adicional.
97         p.setNombre(datos.getNombre());
98         p.setPrecio(datos.getPrecio());
99         p.setStock(datos.getStock());
100        p.setCategoria(datos.getCategoria());
101
102        return p;
103    }
104
105    // DELETE: eliminar un producto por ID
106    public void eliminar(int id)
107    {

```

```

104         Producto p = obtenerPorId(id);
105         productos.remove(p);
106     }
107 }

```

2.5. ProductoNoEncontradoException.java

Listing 7: Clase ProductoNoEncontradoException.java (Pre-entrega)

```

1 package exception;
2
3 /**
4  * Excepcion personalizada lanzada cuando un producto buscado por
5  * su ID
6  * no existe en el sistema.
7  *
8  * Hereda de RuntimeException (excepciones no verificadas). Esto
9  * evita obligar
10 * al desarrollador a usar bloques try/catch de forma explicita,
11 * manteniendo
12 * el codigo mas limpio, pero permitiendo su captura cuando sea
13 * necesario.
14 *
15 * El uso de excepciones propias mejora la expresividad del modelo
16 * de dominio,
17 * ofreciendo nombres claros en lugar de utilizar clases genericas
18 * como
19 * Exception o IllegalArgumentException.
20 */
21 public class ProductoNoEncontradoException extends RuntimeException
22 {
23     public ProductoNoEncontradoException(String mensaje)
24     {
25         // super llama al constructor de la clase padre (
26         // RuntimeException),
27         // la cual almacena el mensaje y lo expone mediante
28         // getMessage().
29         super(mensaje);
30     }
31 }

```

2.6. StockInsuficienteException.java

Listing 8: Clase StockInsuficienteException.java (Pre-entrega)

```

1 package exception;
2
3 /**
4  * Excepcion personalizada lanzada cuando se intenta asignar un
5  * stock invalido
6  * (por ejemplo, un valor negativo).
7  *

```

```
7  * Tambien permite gestionar las validaciones del carrito de
   * compras, senalando
8  * los casos en los que un cliente solicita mas unidades de las
   * disponibles
9  * en el inventario actual.
10 */
11 public class StockInsuficienteException extends RuntimeException
12 {
13     public StockInsuficienteException(String mensaje)
14     {
15         // Llama al constructor de RuntimeException para almacenar
16         // el mensaje de error
17         super(mensaje);
18     }
19 }
```